Assignment 4:

Maze Traversal

Conner Inman

AUTONOMOUS ROBOTICS

March 16, 2026

Introduction

The goal of Assignment 4: Maze Traversal was to build and program a robot that could autonomously navigate a maze and reach the end without outside assistance. This assignment can be broken down into three parts: building, programming, and demonstration. The robot had two front-mounted wheels and a distance sensor mounted at an angle to detect when the wall moved closer or farther away. The programming implemented functions to keep the robot aligned with the wall and allow it to turn around corners. Finally, the demonstration was the most difficult part of the project, as the tolerances and measurements in the code strictly determined how successful the robot would be.

Mechanical Design

The design we chose consisted of a robot with two front wheels and a sled. As shown in Fig. 14, this was always the intention; however, it evolved into the final design shown in Fig. 15 due to concerns about the size of the maze. As a result, we took extensive measures to reduce the overall size of the robot. The sled design removed the need for rear wheels, further reducing the overall size of the robot. This also allowed the robot to turn around its axis without requiring four wheel motors to coordinate the turn.

As shown in Figs. 1–4, we finalized this design; however, we encountered difficulties when determining how to properly mount the distance sensor. A servo was attached to the bottom of the sensor because we initially planned for the robot to scan boxes and then back out of them. However, we ran out of time to implement this feature and instead relied on manual setup for navigating the maze. In the final product, this motor remained attached but was only used to hold

the sensor at approximately –40 degrees. The bracket used to attach the servo to the sensor was designed so the axis could be adjusted at multiple points, allowing us to determine the optimal angle for scanning the wall. Overall, this design proved helpful, as we quickly identified the optimal angle. These axes were then locked in place using screws and nuts to prevent vibrations or movement from shifting the sensor away from its optimal angle.

Software Design

The software for this robot was designed to complement the robot's mechanical design. This includes movement distances and turning routines that account for tail swing during turns. As shown in Figs. 5 and 7, the robot follows a linear algorithm: initialize, find the wall, align with the wall, and then run the main loop. The main loop runs continuously, taking a new distance reading every half millisecond. It then checks a Boolean condition: if the distance is greater than 10.8 inches, the robot stops, drives forward six inches, and executes the ChickenWrap() function (Fig. 10). If this condition is not met, the program runs the tracer() function. This process continues until the maze is completed and the robot is turned off.

A key dependency for this code functioning correctly is the chosen tolerance values. As is shown all through the code we rely on these for many things. The final target distance was 8 inches from the wall, with a tolerance of ±1 inch. These tolerances came to a head during the final demonstration of the bot when they had been made tighter resulting in the bot's failure to find walls after a wrap. A tolerance of one inch was ideal because it allowed the robot to detect the wall at up to 9 inches and then adjust back to its target distance. When this tolerance was reduced to a quarter inch for testing, it effectively reduced the robot's usable detection range. This also resulted in slower run times as the bot had to adjust more often to remain in tolerance.

The ChickenWrap() function was created to move around a corner using a complex hard-coded drive sequence, as illustrated in Fig. 20. Afterward, the robot checks whether a wall is present. If a wall is detected, it continues driving; otherwise, it performs a 90-degree turn, moves forward slightly, and then checks again before aligning with the wall.

The findWall() (Fig. 8) and alignWall() (Fig. 9) functions are closely related functions. They are used for the robot to first locate a wall and then move within the desired tolerance distance. However, findWall() is far less precise than alignWall(), as demonstrated in Fig. 18. This means that findWall() is only called at the beginning of the program, because calling it later could risk a collision with the wall. AlignWall(), however, is called frequently because it keeps the robot following the wall within the specified tolerance. This function also contains a failsafe, as there is a scenario where the robot may become stuck and continuously attempt to adjust its position. This issue was resolved by limiting the adjustment loop to five iterations, after which the loop breaks and allows the tracer() function to call turn().

The tracer() function (Fig. 12) acts as the primary decision-making component within the code. This function follows the flow outlined in Fig. 6. The robot first checks whether the measured distance is smaller than the target by two inches (placing it outside the tolerance range), in which case it executes turn(). If this condition is not met, the robot determines whether it needs to adjust left or right and then drives forward accordingly.

Performance Evaluation

The robot's performance was mixed. On one hand, we successfully demonstrated that a minimal robot with one sensor and two wheels could traverse a maze. However, our main limitation was the selection of measurements and tolerance values. None of the major revisions

were due to the functions themselves. Rather, they resulted from reducing the size of the robot by finding smaller axles to minimize its profile. Adjusting tolerance values also allowed for either finer or looser tracing of the maze walls.

Simply put, our process of testing each function individually ensured that many iterations were completed before integrating all functions together. However, our major failure occurred during the in-class demonstration, when the tolerance values had been significantly reduced. This caused several of the robot's functions to fail because the robot could no longer reliably detect the wall. As shown in Fig. 19, the tolerances were later reverted to their previous values, which allowed the robot to detect walls more easily and follow them more effectively.

Conclusion

The robot's code could have benefited from additional refinement. We were running into issues that would have been worked out with more time. As evidenced by the servo mounted under the sensor, there was a plan to implement a system that would allow the sensor to actively scan for walls or boxes rather than relying solely on the robot's movement. Ultimately, we ran out of time to implement these features and instead focused on resolving other issues and refining the existing system. Another major issue was communication between group members. The target distance and tolerance values were modified while one group member was absent, and it was not communicated that those values had already been determined to be optimal. This resulted in the values being changed shortly before the demonstration. Another challenge we encountered was the maze itself. From week to week, the maze walls varied in width, forcing us to attempt a one-size-fits-all solution. In hindsight, we could have used looser measurements because the final maze had wider walls. This would likely have allowed the robot to make its

movements more accurately. However, beyond these challenges, once the final designs and ideas had been implemented, our primary limitations were time and communication.

Appendix

Figure 1: Front view of the completed robot showing the two-wheel drive configuration and front-mounted distance sensor.

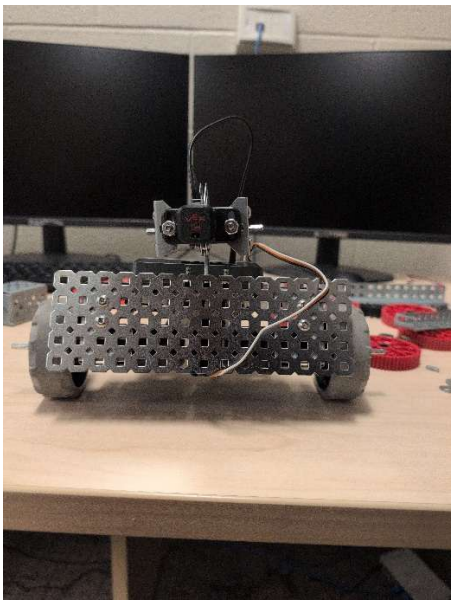

Figure 2: Side view of the robot illustrating the sled support and the angled mounting of the distance sensor.

Figure 3: Rear view of the robot highlighting the sled design used in place of rear wheels.

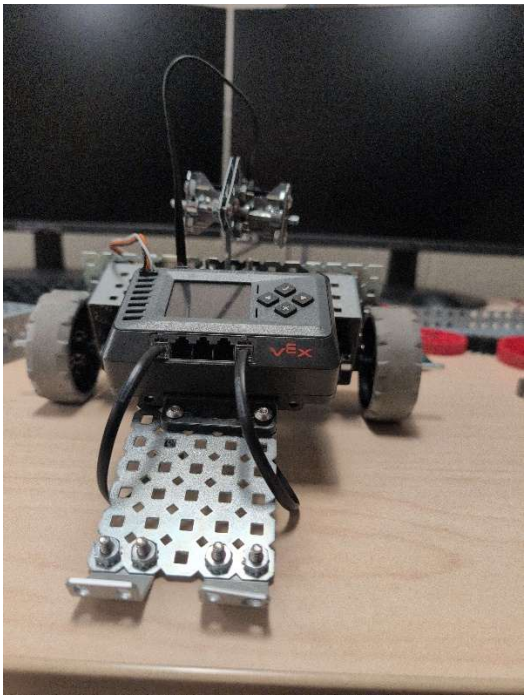

Figure 4: Distance sensor mounting system showing the servo mount and adjustable bracket used to control the sensor angle.

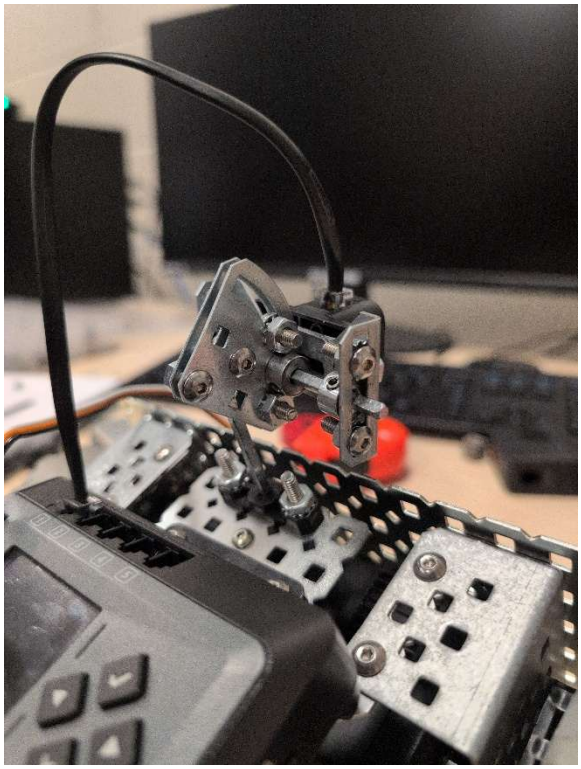

Figure 5: Flowchart illustrating the logic of the robot's main control loop.

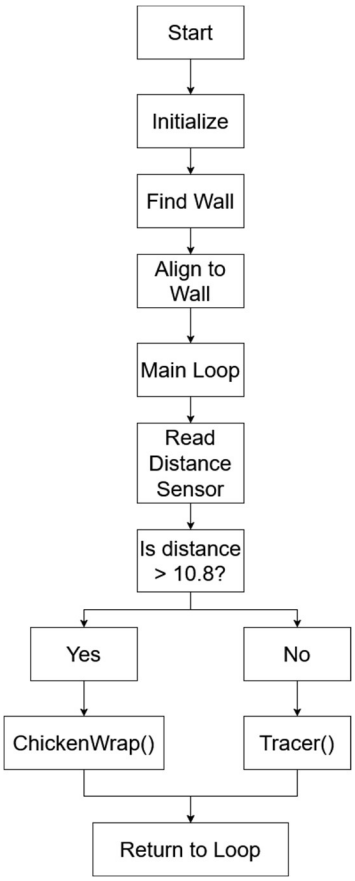

Figure 6: Flowchart showing the decision logic used in the tracer() function for wall following.

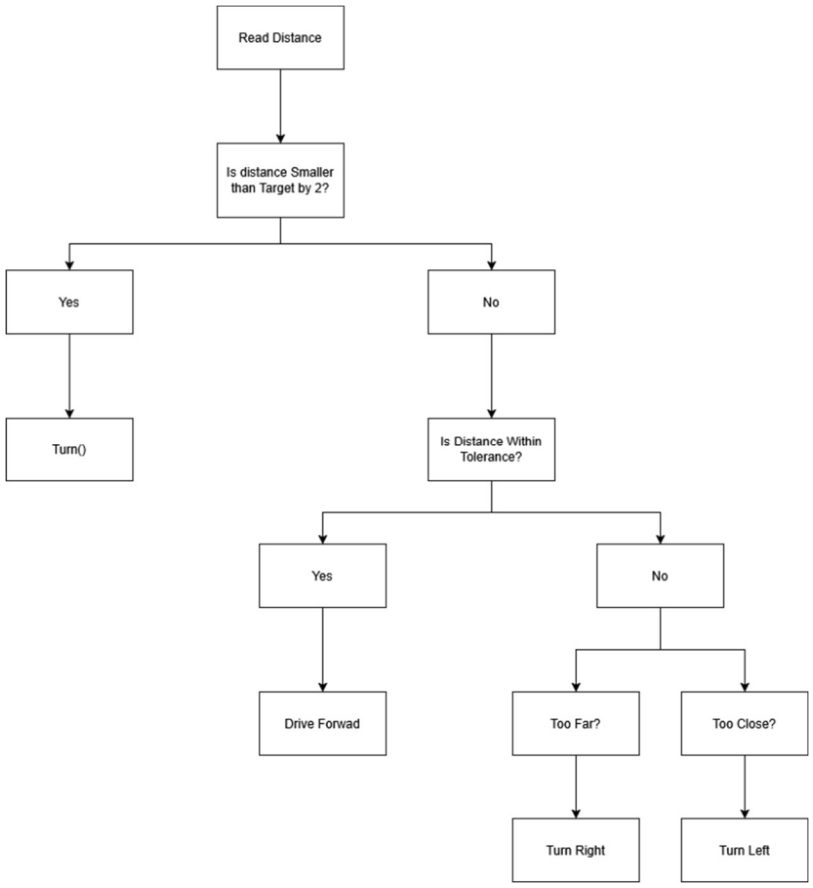

Figure 7: Pseudocode describing the overall maze traversal algorithm.

```
PseudoCode of Maze Traversal

START

Initialize robot
Set target distance = 8 inches
Set tolerance = 1 inch

Move forward
Find nearest wall

Align robot to correct distance from wall

LOOP forever

    Read distance sensor

    Drive forward

    IF distance > large gap threshold
        Stop robot
        Move forward
        Perform corner navigation (chickenWrap)

    ELSE
        Perform wall tracing adjustment
            IF too far from wall
                Turn slightly toward wall
            ELSE IF too close
                Turn slightly away
            ELSE
                Continue forward

END LOOP
```

Figure 8: Implementation of the findWall() function used to locate the nearest wall at startup.

```
def findWall():
    #original 15
    drivetrain.set_drive_velocity(30,PERCENT)
    print("getting tolerant")


    #find the wall
    for i in range(40): # safety limit

        distance = distance_1.object_distance(INCHES)

        if distance < 8: # wall detected
            break

        drivetrain.turn_for(RIGHT, 4, DEGREES)
        drivetrain.drive_for(FORWARD,5,INCHES)
        wait(20, MSEC)
```

Figure 9: Implementation of the alignWall() function used to adjust the robot's distance from the wall within the specified tolerance.

```
def alignWall():
    #original 5
    drivetrain.set_drive_velocity(30,PERCENT)
    #fine tune placement on wall
    # changed from 20 to 5 in case of frick up
    for i in range(5):
        distance = distance_1.object_distance(INCHES)

        if abs(distance - target) <= tolerance:
            break

        if distance > target:
            #too far from wall -> turn right slightly
            drivetrain.set_drive_velocity(20, PERCENT)

            drivetrain.turn_for(RIGHT, 3, DEGREES)

            #original 1
            drivetrain.drive_for(FORWARD, 3,INCHES)
            print("adjusting right")

        elif distance < target:

            drivetrain.turn_for(LEFT, 3, DEGREES)
            #original 1
            drivetrain.drive_for(FORWARD, 3, INCHES)
            print("adjusting left")
```

Figure 10: Implementation of the ChickenWrap() function used to maneuver the robot around maze corners.

```
def chickenWrap():
    drivetrain.stop()
    drivetrain.drive_for(FORWARD,1, INCHES)
    drivetrain.set_turn_velocity(30,PERCENT)
    # drivetrain.turn_for(RIGHT, 90,DEGREES)
    #eric, instead of turning 90, we turn 45, 2 times
    drivetrain.turn_for(RIGHT, 45, DEGREES)
    drivetrain.drive_for(FORWARD, 8, INCHES)
    drivetrain.set_turn_velocity(30,PERCENT)
    drivetrain.turn_for(RIGHT, 45, DEGREES)
    drivetrain.drive_for(FORWARD, 10, INCHES)


    drivetrain.set_turn_velocity(45,PERCENT)
    wait(0.01 , SECONDS)
    distance1 = distance_1.object_distance(INCHES)
    print(distance1)
    if distance1 > 9:
        drivetrain.stop()
        drivetrain.set_turn_velocity(45,PERCENT)
        drivetrain.turn_for(RIGHT, 90,DEGREES)
        print("turning")

    drivetrain.drive_for(FORWARD, 7.5,INCHES)
    alignWall()
    wait(200, MSEC)

    print("wrap")
```

Figure 11: Implementation of the turn() function used to rotate the robot during navigation.

```python
def turn():
    drivetrain.stop()

    #eric
    drivetrain.drive_for(REVERSE, 1, INCHES)
    drivetrain.turn_for(RIGHT, 50 ,DEGREES)
    drivetrain.drive_for(REVERSE, 3, INCHES)
    drivetrain.turn_for(LEFT, 50, DEGREES)
    ## adjust this for fine tuning!!!! yippeee!
    #commenting this out because i dont need it nomo
    # drivetrain.drive_for(REVERSE, 5, INCHES)
    drivetrain.turn_for(LEFT, 80 ,DEGREES)
    # drivetrain.drive_for(REVERSE, 2,INCHES)

    for i in range(20):
        distance = distance_1.object_distance(INCHES)

        if abs(distance - target) <= tolerance:
            break

        if distance > target:
            #too far from wall -> turn right slightly
            drivetrain.turn_for(RIGHT, 3, DEGREES)
            drivetrain.drive_for(FORWARD, .5,INCHES)


        elif distance < target:
            #too close -> turn left slightly
            drivetrain.turn_for(LEFT, 3, DEGREES)
            drivetrain.drive_for(FORWARD, .5,INCHES)

    wait(20, MSEC)

    print("turn")
```

Figure 12: Implementation of the tracer() function responsible for real-time wall-following adjustments.

```python
def tracer(distance):
    print("tracing wall")

    if abs(distance - target) <= tolerance:
        adj= False


    if distance < target - 2:
        #if the distance is smaller than 2 a wall is there and we turn 90

        turn()
        adj = False

    elif distance > target + tolerance :
        #too far from wall -> turn right slightly
        drivetrain.turn_for(RIGHT, 3, DEGREES)

        print("adjusting right")


    elif distance < target - tolerance :
        #too close -> turn left slightly
        drivetrain.turn_for(LEFT, 3, DEGREES)

        print("adjusting left")

    return
```

Figure 13: Source code for the robot's main control loop.

```python
# setting the servo
COUNT = 0
servo_a.set_position(-40, DEGREES)
wait(0.5,SECONDS)
drivetrain.drive_for(FORWARD,8,INCHES)
findWall()
alignWall()

while True:

    distance = distance_1.object_distance(INCHES)

    brain.screen.clear_screen()
    brain.screen.set_cursor(1,1)
    brain.screen.next_row()
    brain.screen.print("Distance : ", distance)
    # print("Distance : ", distance)
    drivetrain.set_drive_velocity(35, PERCENT)
    drivetrain.drive(FORWARD)

    if distance > 10.8:
        drivetrain.stop()
        # commented out because too much
        # drivetrain.drive_for(FORWARD, 12.5, INCHES)
        drivetrain.drive_for(FORWARD, 6, INCHES)
        chickenWrap()
    else:
        tracer(distance)

    wait(0.5, MSEC)
```

Figure 14: Initial design concept for the robot showing the early layout of wheels and sensors.

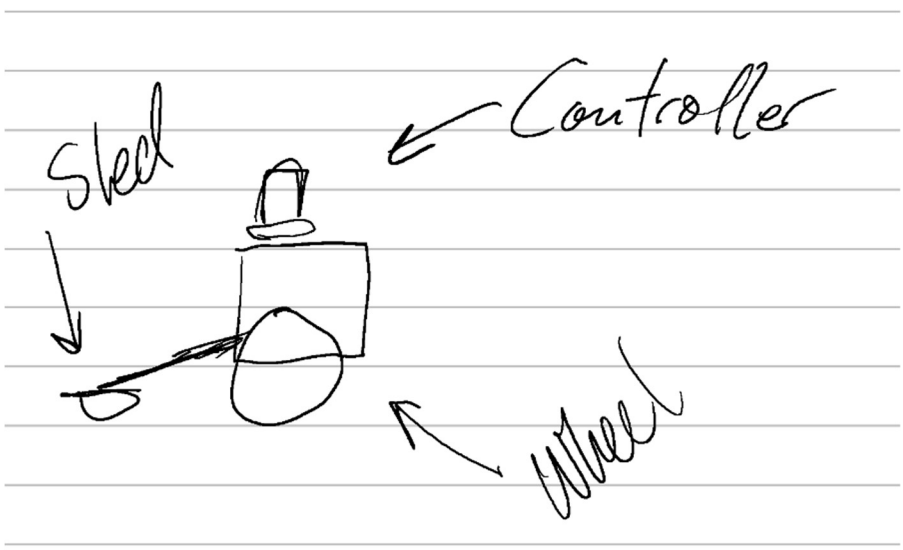

Figure 15: Final design sketch of the robot reflecting size optimizations made for the maze environment.

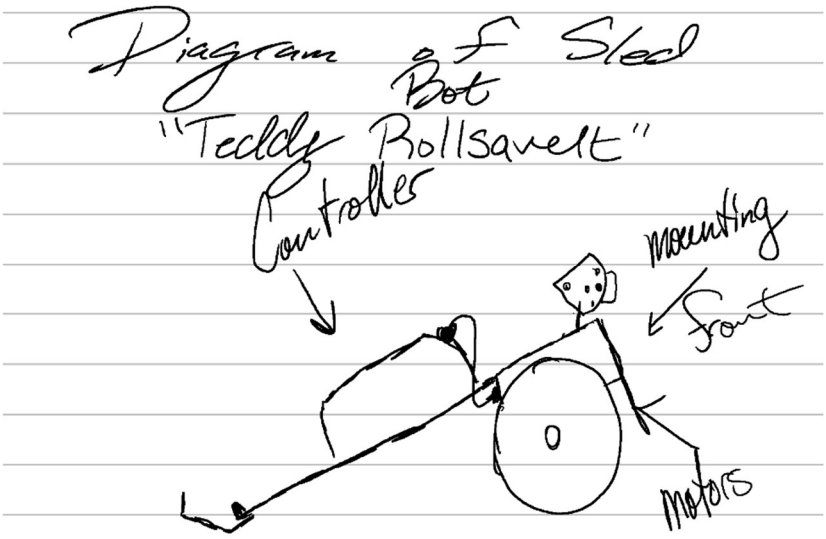

Figure 16: Concept sketch of the distance sensor mounting mechanism.

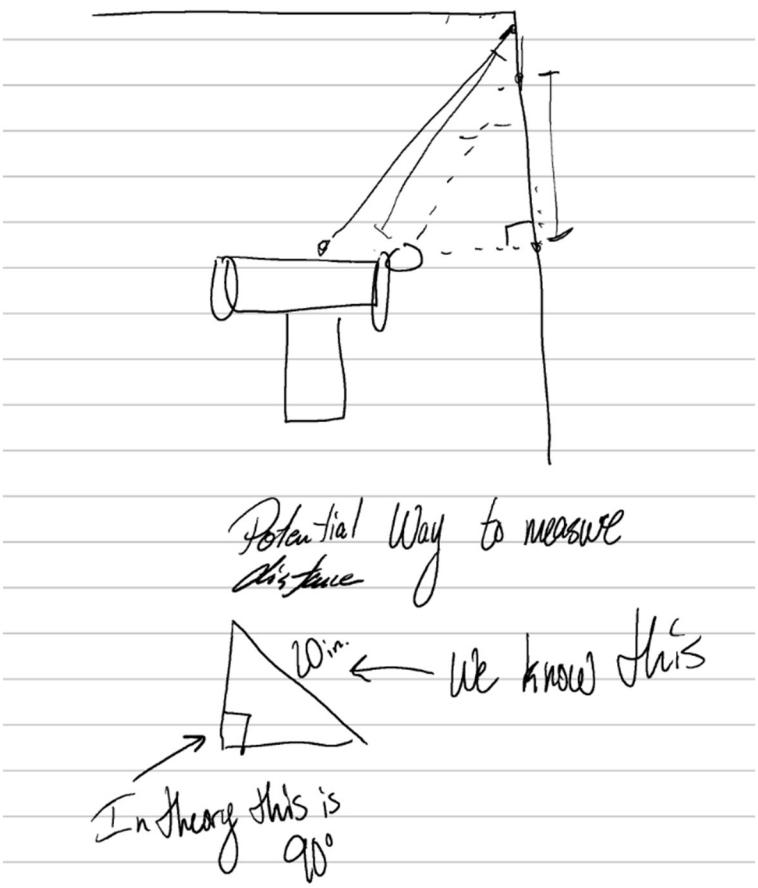

Figure 17: Concept diagram illustrating how the tracer() function interprets distance readings from the wall in front of the robot.

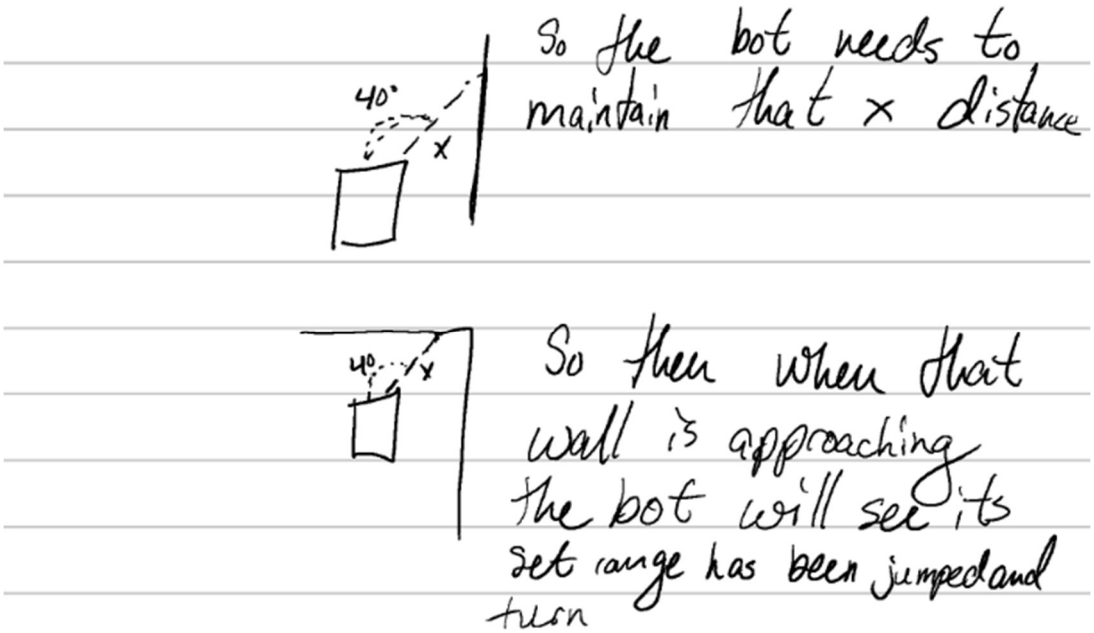

Figure 18: Diagram illustrating the interaction between the main navigation functions.

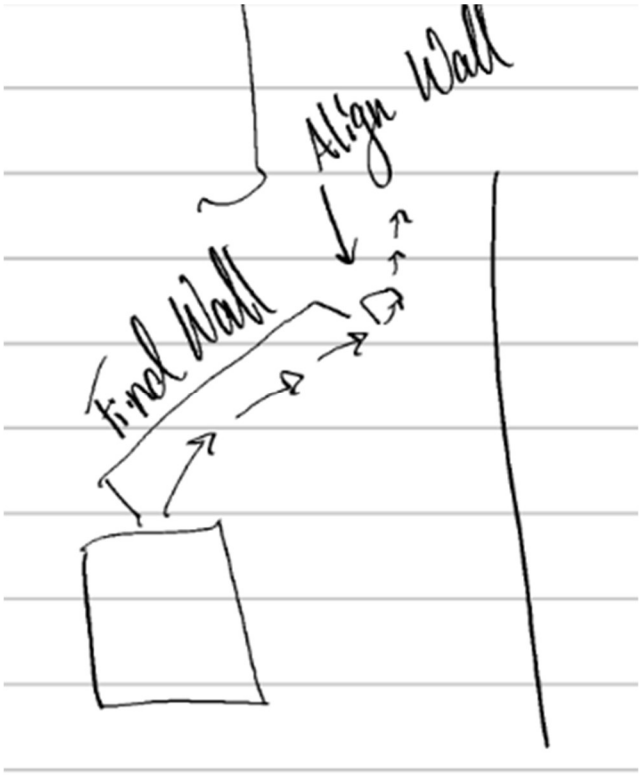

Figure 19: Comparison of tolerance values used during first demonstration and the final demonstration.

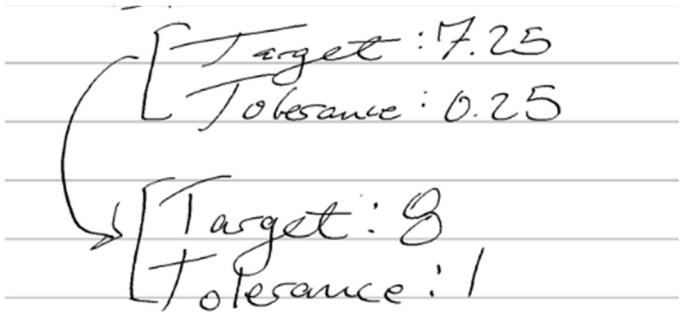
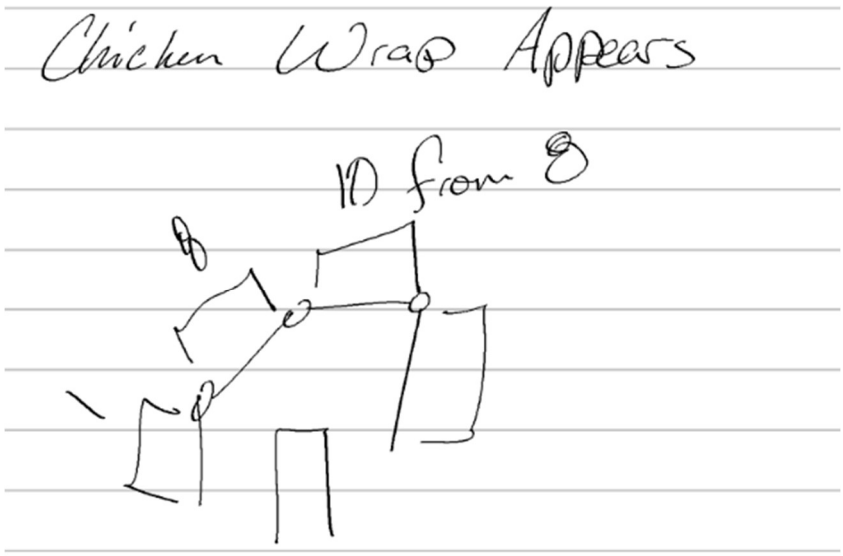
Figure 20: Final sketch illustrating the movement path executed by the ChickenWrap() cornering routine.